CSE 4304-Data Structures Lab. Winter 2021
Date: August 02, 2022
Target Group: All groups
Topic: Arrays, recursion, logic-based problems.

Instructions:

- Task naming format: fullID_T01L01_2A.c/cpp
- If you find any issues in problem description/test cases, comment in the google classroom.
- If you find any test case that is tricky that I didn't include but others might forget to handle, please comment! I'll be happy to add.
- Modified sections will be marked with BLUE color.

Group	Tasks
1B, 2A, 2B	**1 2 3 4**
1A	**3 5 6 7**

Task-01:

Assume your program will take a set of positive integers(1~9) as input and store them in an array. It will stop taking input for any negative number or zero.
Now, the program will show how many times each of the numbers have occurred. (Don't show anything for a number that didn't occur.)

Input	**Output**
3 4 5 7 8 9 2 3 3 4 4 5 0	3 occurs 3 times 4 occurs 3 times 5 occurs 2 times 7 occurs 1 time 8 occurs 1 time 9 occurs 1 time 2 occurs 1 time
5 2 2 3 3 3 1 1 9	3 occurs 3 times 2 occurs 2 times 1 occurs 2 times 5 occurs 1 time 9 occurs 1 time

[**Note**: Output is arranged based on the occurrence of each number. If the frequency of two numbers is the same, it's not mandatory to sort based on the relative ordering.]

Note: STL libraries are not allowed for this task. Implement the sort function manually.

Task-02:

Write a program that takes a positive integer number(n) as input and calculates the factorial(n!) value of the number.

$$n! = n \times (n-1) \times (n-2) \times \ldots \ldots \times 1$$

The factorial value should be calculated in two different ways using the following function:

- Factorial_using_iteration (int n): returns int
- Factorial_using_recursion (int n): returns int

Input	**Output**
5	120 (using iteration) 120 (using recursion)
10	3628800 (using iteration) 3628800 (using recursion)
0	1 (using iteration) 1 (using recursion)

Task-03

‘Binary-search’ is an algorithm that is based on compare and split mechanism. This algorithm is also known as ‘half-interval search’, ‘logarithmic search’, or ‘binary chop’.
The objective is to search for the position of the target value in a sorted array. It compares the target value with the middle element of the array. If the element is equal to the target element, then the algorithm returns the index of the found element. Otherwise, the searching algorithm uses a half section of that array, based on the comparison value, it uses either the first half (when the value is less than the middle) or the second half (when the value is greater than the middle). Then, it does the same for the next half.

Your task is to implement this algorithm in two ways using the following functions:

- Iterative_binary_search(int x, int array[]): returns index
- Recursive_binary_search(int x, int array[]): returns index

Input	**Output**
0 2 6 11 12 18 34 45 55 99 -1 55	8 (using iteration) 8 (using recursion)
25 33 37 43 55 60 -1 42	-1 (using iteration) -1 (using recursion)
2 3 7 13 15 20 25 33 -1 -1	-1 (using iteration) -1 (using recursion)

[**Note**:

- The input array is given in a sorted sequence.
- **Using iterations** means using a loop inside the function that checks for the equality of the middle element.
- **Using recursion** means the function calls itself again and again.
- Array indexing starts from 0.
- It will stop taking input for any negative number.]

Task 4:

IUT hired a group of workers to build a new dormitory. The energy level of each worker varies.
Each day, they have new tasks in hand where they have to nominate two workers to work in a pair. The task has a defined energy level (let's say, 'n'). So two workers with energy levels n_1, and n_2 have to be selected such that $n=n_1+n_2$.

Your task is to find the number of possible pairs of workers that can be nominated to solve the task.

Notes:

- Assuming the serial of worker starts from 1
- Stop taking input when you get 0
- The energy level of workers can be positive/negative (except zero).
- In the output, you have to show the number of pairs and the workers who form the pairs. (worker1,worker-2) means the same pair as (worker-2,worker-1) so don't consider both of them!

Input	**Output**
1 5 7 -1 0 6	2 Pairs: (worker-1, worker-2) (worker-3, worker-4)
1 5 7 -1 5 0 6	3 Pairs: (worker-1, worker-2) (worker-3, worker-4) (worker-1, worker-5)
1 1 1 1 0 2	6 pairs: (worker-1, worker-2) (worker-1, worker-3) (worker-1, worker-4) (worker-2, worker-3) (worker-2, worker-4) (worker-3, worker-4)
10 12 15 -1 7 6 5 4 2 1 1 1 0 11	6 pairs (worker-1, worker-10) (worker-1, worker-11) (worker-1, worker-12) (worker-2, worker-4) (worker-5, worker-8) (worker-6, worker-7)
1 5 3 7 -1 0	2 Pairs:

6	(worker-1, worker-2) (worker-4, worker-5)
1 5 7 -1 0 5	No pair found!

Task 05:
Given two integer arrays, num1, and num2, return an array of their intersection. Each element in the result must appear as many times as it shows in both arrays, and you may return the result in any order.
(keep input for an array until you get -1)

Input	**Output**	**Clarification**
1 2 2 1 -1 2 2 -1	2 2	
4 9 5 -1 9 8 9 8 4 -1	4 9	(9 4) is also acceptable.

Task 06:
The Fibonacci sequence is a series where the next term is the sum of the previous two terms. The first two terms of the Fibonacci series are 0 followed by 1.
The Fibonacci sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21 …

Given a number ‘n’, your task is to write a function for calculating the n-th Fibonacci number of the series.
Solve the problem in both **iterative** and **recursive** approaches.

Input	**Output**
10	34
20	4181

Task 07:

Xenia the beginner mathematician is a third-year student at elementary school. She is now learning the addition operation.

The teacher has written down the sum of multiple numbers. Pupils should calculate the sum. To make the calculation easier, the sum only contains digits. Still, that isn't enough for Xenia. She is only beginning to count, so she can calculate a sum only if the summands follow in non-decreasing order. For example, she can't calculate the sum 1+3+2+1 but she can calculate sums 1+1+2 and 3+3.

You've got the sum that was written on the board. Rearrange the terms and print in such a way that Xenia can calculate.

Input	**Output**
3+2+1	1+2+3
1+1+3+1+3	1+1+1+3+3
2	2
5+2+2+4	2+2+4+5